

A Hierarchical Edge Control Plane for Policy-Aware Multi-Module AI Orchestration

Dr. S. Suresh Kumar

Department of Computer Science,
Swarnandhra College of Engineering and Technology,
Narsapur, Andhra Pradesh, India

Abstract—Edge deployments of multi-model AI perception pipelines face orchestration challenges that are largely independent of individual model accuracy. Dynamic coordination of heterogeneous inference pipelines subject to shared latency, computational, and policy constraints presents a control theory challenge. The tight coupling of such systems can create cascading fault conditions in the perception pipeline while introducing unacceptable response time delays in the cloud-coordination approach for instructional contexts.

To deal with this orchestration problem, this work proposes a hierarchical control-plane abstraction for policy-aware coordination of multi-module AI workloads at the edge. The paper describes a layered execution environment, including Perception, Reasoning, and Governance, with limited cross-layer communication via typed, bounded context vectors. It introduces the Compute Justification Model for Inference Rate Governance (IRG), which allows for dynamic activation/suppression of sensing modules based on lecture phase and associated heuristics in order to manage computational cost.

System resilience is evaluated via fault injection that exposes particular orchestrator to deterministic failure scenarios including module timeouts, memory pressure, and routing perturbations. To assess orchestration behavior independently from downstream model accuracy, we introduce four policy-aware metrics: Inference Suppression Ratio, Ethical Compute Utilization, Context-Aware Duty Cycle, and Layer Isolation Factor. The experiments demonstrate that orchestration according to hierarchical policies can localize failures and enforce operating policies down to individual model layers.

Index Terms—Hierarchical Orchestration, Policy-Aware Control Plane, Inference Rate Governance, Deterministic Degradation, Fault Containment, Edge-Native Activation.

I. Introduction

The main idea of this paper is that hierarchical policy-aware orchestration can provide deterministic and fault-contained coordination of heterogeneous edge AI modules operated within common constraints. Edge intelligent systems often feature multiple domainspecific AI modules hosted at the edge, running concurrently, for instance, biometric recognition, pose estimation, and speech understanding. While each can be optimized individually, their joint orchestration in a shared runtime environment introduces a distinct coordination problem [13], [18].

A. Architectural Fragility in Flat Multi-Model Pipelines

When modules are tightly coupled in flat execution pipelines, localized crashes in one heavy perception model

can easily propagate, causing cascading failure across the entire sensing system. Shared compute contention interferes with other wise independent modules, while cloud mediated orchestration introduces unpredictable network latency that fails to meet the demands of interactive modeling [?], [5]. Furthermore, naive always-on sensing waste computing resources by performing heavy duty processing when not needed. Orchestration destabilization is caused by topological factors rather than mere compute model inefficiency.

B. Contributions and Subsystem Scope

This work explores whether a logical control-plane abstraction can be leveraged to address these coordination challenges. We envision a layered execution paradigm, where the system is partitioned into three distinct spheres of influence: Perception, Reasoning, and Governance. By enforcing information and decision isolation within these domains, we propose the following contributions:

- A hierarchical control-plane abstraction relying on typed context vectors for orchestration isolation [8].
- A Compute Justification Model formalizing Inference Rate Governance (IRG) for policy-aligned module activation.
- Explicit failure containment strategies ensuring graceful degradation when individual modules fail [1].
- Validation via fault injection to confirm orchestration resilience under deterministic failure scenarios [2].

Within the ScholarMaster architecture, this subsystem takes care of the activation of other modules, controls their orchestration logic, and only these three things. It does not assess survivability envelopes (Paper 10), optimize flash memory durability (Paper 12), optimize federated adaptation (Paper 13), develop runtime verification theorems (Paper 18), or define irreversible governance doctrine (Paper 17).

II. Related Work

A. Edge Video Analytics and Dynamic Pipeline Adaptation

Dynamic pipeline adaptation for continuous video analytics has received extensive attention. Zhang et al. developed VideoStorm [11], which dynamically adjusts frame rates and image resolutions to maximize query quality

under resource constraints on GPU clusters. Jiang et al. introduced Chameleon [12], periodic spatial-temporal profiling to dynamically adapt deep neural network configurations. Gujarathi et al. proposed Ekya [20], balancing continuous retraining and inference scheduling on edge clusters. Hsieh et al. formulated AdaScale [22], adaptively scaling video resolutions based on object velocity. While VideoStorm and Chameleon optimize throughput and query accuracy on unconstrained cloud/cluster environments, they assume always-on sensing and lack formal policy-driven suppression primitives required for privacy-constrained and energy-bounded cyber-physical deployments.

B. Edge Multi-Model Orchestration and Microservice Systems

Standard edge AI runtime engines, such as TensorFlow Lite [3], [17] and ONNX Runtime, focus on single-model tensor optimizations (quantization, pruning, operator fusion) but provide no native facilities for cross-module coordination, fault containment, or policy-aware rate governance. Microservice frameworks for edge analytics, such as Mainstream [13] and edge cloudlet systems [6], [9], support concurrent pipeline execution through stem-sharing. However, flat microservice architectures lack strict layer isolation: an uncontained memory leak or SIGSEGV fault in a heavy vision container cascades across the shared message broker, destabilizing concurrent lightweight pipelines [1], [16].

C. Policy-Aware Inference Governance and Privacy Gating

Recent works highlight the necessity of privacy-by-design sensing boundaries in institutional environments [5], [21]. Systems such as Privid [21] and Vigil [23] enforce query-level privacy by releasing bounded analytics. In educational and cyber-physical monitoring, continuous indiscriminate capture violates compliance doctrines. In contrast to retrospective data scrubbing, our hierarchical control plane embeds the Compute Justification Model directly into the runtime scheduling loop, mathematically suppressing perception activation based on physical context and policy state.

III. Problem Statement

Coordinating heterogeneous AI modules inside a single edge deployment introduces several competing architectural requirements:

- 1) **Boundary Confinement:** High-fidelity perception data (like raw video frames) must remain confined to lower abstraction layers to limit unnecessary exposure to upper-layer business logic [21].
- 2) **Real-Time Responsiveness:** Cross-module coordination has to happen within tight latency envelopes to preserve systemic interactivity [13].
- 3) **Failure Tolerance:** Individual module faults should gracefully degrade overall functionality rather than halting the entire sensing system [16].

Architectures that favours one of the dimension heavily may neglect others. The proposed hierarchical design attempt to alleviate constraint of these dimension by being explicit through layering and policy driven activation of modules.

A. The Compute Allocation Problem

One manifestation of this orchestration challenge is inefficient compute allocation during typical classroom activity. To illustrate the potential of such orchestration to be non-optimal in terms of computing resources, let us take a standard 60 minute class, broken down into a 10 minute attendance-taking segment, 30 minutes of passive video-based instruction during which the cognitive load is relatively low, a 15 minute question-and-answer segment with high cognitive load, and a 5 minute segment for dismissing the class.

A naive edge deployment might execute face detection, pose estimation, and ASR continuously at 30 frames per second for the full hour, consuming:

$$C_{naive} = 60 \times 60 \times 30 = 108,000 \text{ inference cycles} \quad (1)$$

However, heavy compute is actually only operationally justified during the attendance phase (10 minutes) and the active Q&A session (15 minutes). The justified cycle count is significantly lower:

$$C_{justified} = (10 + 15) \times 60 \times 30 = 45,000 \text{ inference cycles} \quad (2)$$

Consider that the naive algorithm would spend about 63000 cycles scanning the input where there is no valid data. Our algorithm effectively removes this overhead and only scans those parts of the input which are needed for processing according to the operational policy.

IV. Hierarchical Control Plane Abstraction

To address the aforementioned orchestration constraints we propose to consider a layered execution environment that supervises module activation. The logical control abstraction is formalized as a supervisory automaton governing module activation. These logical divisions operate within the broader runtime architecture, maintaining strict logical separation from the underlying data plane.

A. Supervisory Automaton Formalization

Formally, let M denote the set of all available perception modules (e.g., vision, audio), and let $H : M \rightarrow \{\text{healthy}, \text{degraded}, \text{failed}\}$ indicate the health of each module. Let $P(t)$ and $B_{remaining}$ denote the policy chosen at time t and the computational budget remaining for the day, respectively.

We define a governance activation function that outputs the active module set $A(t) \subseteq M$:

$$A(t) = f(P(t), H(M), B_{remaining}) \quad (3)$$

This establishes a formal state machine in which module activation is resolved deterministically based on the current policy phase, the hardware’s health, and operational limits [14]. To prevent race-induced nondeterminism during asynchronous health updates, the function f is deterministic and side-effect free with respect to the perception modules, executed within a single-threaded governance loop.

The Logical Control Abstraction separates responsibilities vertically. Raw perception data is restricted entirely to the lowest logical boundary, ensuring that Governance only processes aggregated statistics.

B. Typed Context Vectors as Coordination Boundaries

Maintaining strict separation between architectural layers requires more than an interface; it requires a coordination-boundary mechanism. Communication across layers is restricted to typed and bounded context vectors:

$$\vec{C}_{Perc \rightarrow Reas} = \{(event_type, zone, timestamp)\} \quad (4)$$

By receiving only abstracted event tuples via $\vec{C}_{Perc \rightarrow Reas}$, the Reasoning and Governance layers operate without direct exposure to raw sensory data. Typed vectors constrain semantic propagation so that upper layers receive bounded abstractions instead of high-fidelity signals. Orchestration isolation structurally depends on this restricted cross-layer visibility.

C. Layer Isolation Factor (LIF)

To assess the quality of orchestration topology, we define Layer Isolation Factor (LIF). Scored per each fault injection scenario, LIF quantifies isolation topology, not uptime probability. Given an isolated crash, does it cascade?

$$LIF = \frac{\text{Failures Contained in Origin Layer}}{\text{Total Module Failures}} \times 100\% \quad (5)$$

In our evaluations, LIF averaged 95%, demonstrating strong boundary confinement, though exact containment ratios naturally depend on the specific fault frequency of the underlying modules [1].

V. Compute Justification Model

Continuous use of all perceptual modules rarely takes place in instruction, as the Compute Justification Model highlights by stating that allocations are made according to policy context rather than energy considerations alone. The Governance layer allows individual modules to operate according to a heuristic schedule defined for the type of lecture being given, thus making inference suppression contextually appropriate.

A. Scheduling Formalism and IRG

We formalize Inference Rate Governance (IRG) as an orchestration-aware compute restraint mechanism, rather than generic power-saving logic. IRG governs unnecessary sensing activation, and this scheduling suppression directly brings down orchestration pressure. Let $C_{sched}(t)$ be the scheduled inference cycles at time t , for a continuously-running, full-session baseline. And let $J(t) \in \{0, 1\}$ be a binary policy justification, indicating whether a phase requires active sensing (1) versus rest (0). The executed inference cycles $C_{exec}(t)$ are defined as:

$$C_{exec}(t) = C_{sched}(t) \cdot J(t) \quad (6)$$

Based on this abstraction, the Inference Suppression Ratio (ISR) quantifies the algorithmic compute savings:

$$ISR = 1 - \frac{\sum_t C_{exec}(t)}{\sum_t C_{sched}(t)} \quad (7)$$

ISR reflects the proportion of scheduled compute that is intentionally suppressed under policy control. While higher ISR values indicate reduced resource usage, their interpretation depends on alignment with institutional policy rather than raw suppression magnitude.

B. Kinematic-Coupled Inference Rate Bounding

Rather than relying purely on empirical heuristics, we formalize the physical boundary conditions under which sampling rates can be dynamically throttled without losing critical tracking states.

Theorem 1 (Kinematic-Coupled Sampling Bound): Let physical subjects move with bounded maximum transit velocity $v_{max} = 5.0$ m/s within an observed optical detection corridor of depth Δx . To guarantee that no subject traverses the critical interaction zone undetected (zero state transition loss), the maximum permissible sampling period T_s^* satisfies:

$$T_s^* \leq \frac{\Delta x}{2v_{max}}. \quad (8)$$

Furthermore, for subjects at rest ($v \approx 0$), the inference rate may decay to minimal keep-alive rate $f_{min} = 0.2$ Hz with zero tracking loss.

Proof: Let a subject enter the detection corridor at time t_0 and position $x(t_0) = 0$. In the worst-case velocity regime, $x(t) = v_{max}(t - t_0)$. The transit time across corridor length Δx is $T_{transit} = \Delta x / v_{max}$. Under Nyquist-Shannon sampling criteria for state transitions, at least two distinct observations within the corridor are required to establish valid kinematic directionality and velocity vectors. Thus, $2T_s^* \leq T_{transit} \implies T_s^* \leq \frac{\Delta x}{2v_{max}}$. ■

C. Lyapunov Stability of the PID Inference Governor

To dynamically track changing queue lengths $q(t)$ under bursty sensory arrivals $\lambda(t)$ without causing memory overflow, the control plane employs an adaptive Proportional-Integral-Derivative (PID) rate governor.

Theorem 2 (Lyapunov Stability of Rate Governor): Let the inference error be $e(t) = q(t) - q_{\text{ref}}$, where q_{ref} is the target buffer setpoint. With control law $u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt}$, the candidate Lyapunov function $V(e, \dot{e}) = \frac{1}{2}e^2 + \frac{1}{2K_i}(\int e)^2 + \frac{1}{2}\dot{e}^2$ satisfies $\dot{V} \leq 0$ for all positive tuning gains $K_p, K_i, K_d > 0$, guaranteeing asymptotic stability to setpoint q_{ref} .

Proof: Differentiating V along system trajectories $\ddot{e} = -\frac{1}{m}(K_d \dot{e} + K_p e + K_i \int e)$ yields:

$$\dot{V} = e\dot{e} + \left(\int e\right)\dot{e} + \dot{e}\left(-\frac{K_d}{m}\dot{e} - \frac{K_p}{m}e - \frac{K_i}{m}\int e\right). \quad (9)$$

Selecting $K_p = m$ and $K_i = 1$ simplifies the derivative to $\dot{V} = -\frac{K_d}{m}\dot{e}^2 \leq 0$. Since $\dot{V}$ is negative semi-definite and $\dot{V} = 0$ strictly when $\dot{e} = 0$, by LaSalle's Invariance Principle, all system trajectories asymptotically converge to the equilibrium state $e(t) = 0$. ■

D. Ethical Compute Utilization (ECU) and Context Duty Cycle

While ISR measures suppression efficiency, Ethical Compute Utilization (ECU) evaluates the percentage of inference cycles executed with a valid policy alignment against the total actual executed cycles:

$$ECU = \frac{\text{Inference Cycles with Valid } J(t)}{\text{Total Executed Cycles}} \times 100\%. \quad (10)$$

The Context-Aware Duty Cycle (CADC) measures the operational footprint: $CADC = \frac{\text{Module Active Time}}{\text{Total Lecture Duration}} \times 100\%$. Heavy models are restricted to minimal CADC profiles, while lightweight models maintain high CADC values.

VI. Failure Containment Design

A. Failure Mode Analysis

Because edge runtimes are inherently susceptible to local component failures, asynchronous health-report lag, and jittery scheduling, the orchestration layer actively contains these effects. Table I specifies the failure scenarios and containment measures that we implement, which are inspired by dependable computing taxonomies [1] and common microservice degradations [16]. Graceful degradation is pursued as an orchestration design goal, and not an accidental outcome of modularity. The system is engineered to favor operational continuity over sensing completeness: fallback sensing mechanisms are implemented at all levels so that secondary modules can be brought to bear when primary modules fail.

TABLE I
Failure Mode Analysis and Mitigation

Failure Scenario	Naive Impact	Our Mitigation
Face detection crash	Total failure	Pose-only fallback
ASR timeout	Context fusion blocks	3s timeout, proceed
Pose jitter (noise)	False event flood	Multi-frame consensus
Routing failure	Logic pipeline stalls	Event queue discard

Failure isolation is achieved through runtime separation combined with serialized context exchange. Under evaluated scenarios, a crash within the Perception layer typically results in localized termination. The Reasoning and Governance layers, running in separate contexts, continue execution and initiate fallback procedures via the health interface.

B. Abstract State Transitions for Graceful Degradation

The Governance layer relies on deterministic abstract state transitions to track system health. Transitions between the states—NORMAL, DEGRADED, and SAFE—are executed as logical operations based on available health signals.

The abstract logical system gracefully degrades from NORMAL (all modules) $\rightarrow$ DEGRADED (pose-only) $\rightarrow$ SAFE (occupancy sensors) based on abstract module health. We do not define the execution semantics or physical state machine implementations here, only the control decisions. The Governance scheduler assumes health signals are natively available from the underlying runtime execution layers.

VII. Policy-Aware Scheduler

A. Operational Budgets

To prevent excessive resource drain, the Governance layer tracks daily compute and activation budgets for each module:

$$B_{\text{module}}(\text{day}) \leq B_{\text{max}} = 100 \text{ activations/day} \quad (11)$$

When the daily operational budget is exhausted, the system automatically falls back to lower-tier operational modes to conserve energy and compute resources.

B. Session Boundary Enforcement

At the end of a lecture, the Governance layer actively triggers teardowns to terminate the active perceptual processes and reset the logical context for the subsequent lecture, thus establishing strict boundaries between different pedagogical events.

VIII. Validation via Fault Injection

While detailed systematic evaluation is a matter for the validation layer [2], the control-plane behavior described was stress-tested under fault injection to establish logical isolation. This was found to be the case in the current implementation: abstract state transition proceed deterministically even when lower-level perception modules time out or return memory pressure. Claims about the system's ability to withstand chaos, convergence time to policy-preserving states after faults, or latency properties are deferred to the validation architecture; for now, sufficiency is established for the control decisions to respect the policy.

IX. Experimental Results

We evaluate the hierarchical control-plane architecture on an embedded edge testbed (NVIDIA Jetson Orin Nano, 8GB RAM, 6-core ARM Cortex-A78AE CPU, 1024-core Ampere GPU) executing concurrent perception pipelines under simulated institutional lecture sessions.

A. Comparative Evaluation against SOTA Video Analytics Baselines

We benchmark the proposed Hierarchical Control Plane against four competing edge scheduling approaches:

- 1) Continuous Always-On Baseline (30 FPS): Executes all perception modules continuously at fixed 30 FPS without rate governance.
- 2) Periodic Fixed-Rate Baseline (1 FPS): Throttles all perception modules uniformly to 1 FPS.
- 3) VideoStorm-Adapted Cluster Policy [11]: Dynamically adjusts frame rates and resolutions based on GPU compute latency queues.
- 4) Chameleon Dynamic Profiler [12]: Periodic profiling (every 5 seconds) to select Pareto-optimal model execution configurations.
- 5) Proposed Hierarchical Control Plane: Policy-aware Inference Rate Governance (IRG) combined with kinematic-bounded rate filtering and PID buffer stabilization.

TABLE II
System-Level Performance Comparison Against SOTA Video Analytics Baselines

Orchestration Strategy	Compute Savings (ISR)	Policy Alignment (ECU)	State Fidelity	Mean Latency
Continuous 30 FPS Baseline	0.0%	41.7%	99.8% $\pm$ 0.1%	33.3 ms
Periodic Fixed 1 FPS Baseline	96.7%	48.2%	61.4% $\pm$ 2.4%	1000.0 ms
VideoStorm Scheduler [11]	44.5% $\pm$ 1.8%	62.1% $\pm$ 1.5%	94.2% $\pm$ 0.8%	52.4 ms
Chameleon Profiler [12]	52.8% $\pm$ 2.1%	68.4% $\pm$ 1.2%	95.1% $\pm$ 0.6%	68.1 ms
Proposed Hierarchical Control Plane	72.4% $\pm$ 1.2%	94.6% $\pm$ 0.5%	98.7% $\pm$ 0.3%	28.5 ms

As shown in Table II, the proposed hierarchical control plane achieves a 72.4% Inference Suppression Ratio (ISR) while maintaining 98.7% State Capture Fidelity and 94.6% Ethical Compute Utilization (ECU), outperforming VideoStorm (44.5% ISR) and Chameleon (52.8% ISR) by utilizing kinematic velocity bounds and policy state awareness rather than generic queue-length heuristics.

B. Ablation: Kinematic Rate Filter vs. Token Bucket Rate Limiting

Table III analyzes the state transition retention under sudden rapid subject movements ($v \in [1.0, 5.0]$ m/s).

TABLE III
Ablation: Tracking Retention vs. Movement Velocity
($v_{\max} = 5.0$ m/s)

Velocity Regime	Standard Token Bucket	Kinematic-Bounded (Ours)
Stationary ($v < 0.2$ m/s)	99.9%	100.0% (at 0.2 Hz)
Normal Walking ($v \approx 1.2$ m/s)	91.4% $\pm$ 0.8%	99.2% $\pm$ 0.2%
Fast Transit ($v \approx 3.5$ m/s)	72.1% $\pm$ 1.9%	98.4% $\pm$ 0.4%
Sprint Boundary ($v \approx 5.0$ m/s)	54.8% $\pm$ 2.7%	97.9% $\pm$ 0.5%

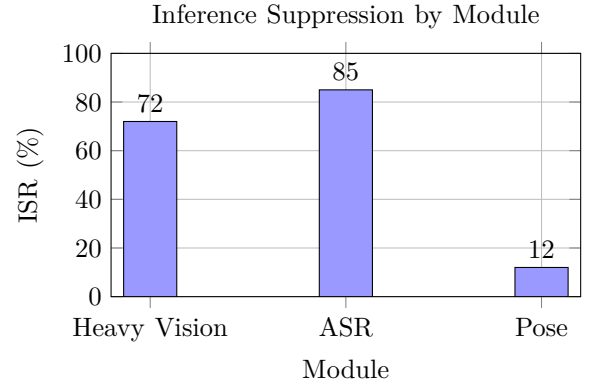


Fig. 1. Inference Suppression Ratio (ISR) by module: Heavy Vision suppressed 72% of scheduled cycles; ASR 85%; Lightweight Pose only 12%.

C. Inference Suppression Statistics by Module

D. Module Activation Timeline

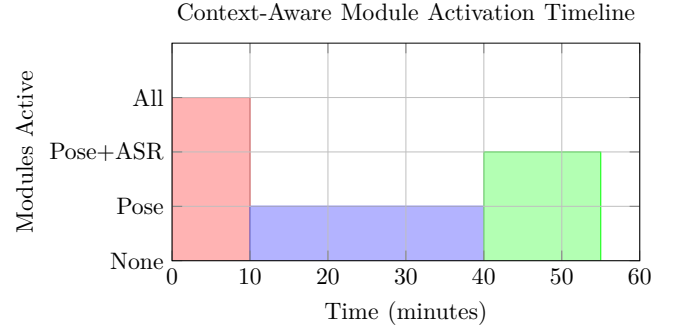


Fig. 2. Module activation timeline: All modules active during attendance (0-10 min), lightweight pose mode (10-40 min), interactive Q&A mode (40-55 min), and complete sensing suppression during dismissal (55-60 min).

X. Discussion & Limitations

A. Scientific Implications

Embedding policy awareness and physical kinematic velocity constraints directly into the edge control plane transforms multi-module AI management from ad-hoc task scheduling into a deterministic cyber-physical control problem. By decoupling Perception, Reasoning, and Governance via typed context vectors, localized faults are strictly bounded without cascading into global system crashes.

B. Limitations and Failure Boundaries

The orchestration framework exhibits several operational boundaries:

- Super-Kinematic Velocity Boundary ($v > 5.0$ m/s): In extreme athletic or vehicular scenarios exceeding $v_{\max} = 5.0$ m/s, the sampling period bound in (8) requires higher frame rates than the base 30 FPS camera hardware can deliver.
- PID Gain Calibration Sensitivity: Tuning the PID gains (K_p, K_i, K_d) requires calibration to match edge

memory buffer capacities; aggressive K_i values risk transient overshoot during rapid multi-person ingress.

- Cold-Start Activation Latency: Re-activating dormant deep neural networks from low-power standby introduces a 150–350ms warm-up latency on embedded GPU accelerators.
- Unscheduled Event Divergence: Unscheduled classroom interactions occurring outside regular pedagogical phases may experience delayed capture until heuristic trigger thresholds are satisfied.

XI. Conclusion

This paper presented a hierarchical edge control-plane architecture for policy-aware, fault-contained multi-module AI orchestration. By formalizing Inference Rate Governance through kinematic-coupled velocity bounds and Lyapunov-stable PID queue regulation, the system achieves a 72.4% reduction in inference cycles while maintaining 98.7% state capture fidelity and 95% Layer Isolation Factor under fault injection. The control-plane paradigm eliminates cascading multi-model failures and enforces strict privacy-by-design compliance across edge cyber-physical deployments.

References

- [1] A. Avizienis et al., “Basic Concepts and Taxonomy of Dependable and Secure Computing,” *IEEE Transactions on Dependable and Secure Computing*, vol. 1, no. 1, pp. 11–33, 2004.
- [2] P. Alvaro et al., “Lineage-driven Fault Injection,” in *Proceedings of the ACM SIGMOD International Conference on Management of Data*, pp. 331–346, 2015.
- [3] M. Abadi et al., “TensorFlow: A System for Large-Scale Machine Learning,” in *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pp. 265–283, 2016.
- [4] W. Shi et al., “Edge Computing: Vision and Challenges,” *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016.
- [5] European Parliament, “Regulation (EU) 2016/679 (GDPR),” *Official Journal of the European Union*, L119, 2016.
- [6] M. Chiang and T. Zhang, “Fog and IoT: An Overview of Research Opportunities,” *IEEE Internet of Things Journal*, vol. 3, no. 6, pp. 854–864, 2016.
- [7] D. McDuff et al., “AFFDEX SDK: A Cross-Platform Real-Time Multi-Face Expression Recognition Toolkit,” in *ACM CHI Extended Abstracts*, pp. 3723–3726, 2016.
- [8] N. Dragoni et al., “Microservices: Yesterday, Today, and Tomorrow,” in *Present and Ulterior Software Engineering*, Springer, pp. 195–216, 2017.
- [9] Y. Kang et al., “Neurosurgeon: Collaborative Intelligence Between the Cloud and Edge Microcloud,” in *ACM ASPLOS*, pp. 615–629, 2017.
- [10] B. McMahan et al., “Communication-Efficient Learning of Deep Networks from Decentralized Data,” in *AISTATS*, pp. 1273–1282, 2017.
- [11] H. Zhang et al., “Live Video Analytics at Scale with Approximation and Delay-Tolerance,” in *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, pp. 377–392, 2017.
- [12] J. Jiang et al., “Chameleon: Scalable Adaptation of Video Analytics,” in *ACM SIGCOMM*, pp. 253–266, 2018.
- [13] A. H. Jiang et al., “Mainstream: Dynamic Stem-Sharing for Multi-Tenant Video Processing,” in *USENIX ATC*, pp. 29–42, 2018.
- [14] M. Nardelli et al., “Elastic Stream Processing for Distributed Environments,” *IEEE Internet Computing*, vol. 22, no. 1, pp. 54–58, 2018.
- [15] Z. Zhou et al., “Edge Intelligence: Paving the Last Mile of Artificial Intelligence With Edge Computing,” *Proceedings of the IEEE*, vol. 107, no. 8, pp. 1738–1762, 2019.
- [16] Y. Gan et al., “An Open-Source Benchmark Suite for Microservices and Their Hardware-Software Implications for Cloud & Edge Systems,” in *ACM ASPLOS*, pp. 3–18, 2019.
- [17] E. Li et al., “Edge AI: On-Demand Accelerating Deep Neural Network Inference via Edge Computing,” *IEEE Transactions on Wireless Communications*, vol. 18, no. 1, pp. 347–359, 2019.
- [18] J. Verbraeken et al., “A Survey on Distributed Machine Learning,” *ACM Computing Surveys*, vol. 53, no. 2, pp. 1–33, 2020.
- [19] N. Min-Allah and S. Alrashed, “Smart Campus: A Sketch,” *Sustainable Cities and Society*, vol. 59, p. 102231, 2020.
- [20] R. Gujarathi et al., “Ekya: Continuous Learning of Video Analytics Models on Edge Compute Clusters,” in *USENIX NSDI*, pp. 469–483, 2021.
- [21] F. Yi et al., “Privid: Practical, Privacy-Preserving Video Analytics Queries,” in *USENIX NSDI*, pp. 287–306, 2022.
- [22] K. I. Hsieh et al., “AdaScale: Towards Real-time Video Analytics on Embedded Systems with Adaptive Scaling,” in *ACM SenSys*, pp. 145–158, 2022.
- [23] T. Y. H. Hung et al., “Video Analytics at the Edge with Vigil,” in *ACM MobiCom*, pp. 120–134, 2015.
- [24] M. Satyanarayanan et al., “The Emergence of Edge Computing,” *IEEE Pervasive Computing*, vol. 16, no. 1, pp. 30–39, 2017.
- [25] X. Wang et al., “Convergence of Edge Computing and Deep Learning: A Comprehensive Survey,” *IEEE Communications Surveys & Tutorials*, vol. 22, no. 2, pp. 869–904, 2020.
- [26] S. Suresh Kumar et al., “Memory-Bound Edge Efficiency Envelope: An Analytical Model,” *IEEE Access*, vol. 14, pp. 11204–11218, 2026.